

Task 2_ Part 2

Importing libraries

```
▼ TASK 2

[ ] import pandas as pd
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score, f1_score
```

Installing autogluon

```
ⓧ pip install autogluon

Requirement already satisfied: autogluon in /usr/local/lib/python3.11/dist-packages (1.2)
Requirement already satisfied: autogluon.core==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.core[all]==1.2->autogluon) (1.2)
Requirement already satisfied: autogluon.features==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon) (1.2)
Requirement already satisfied: autogluon.tabular==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.tabular[all]==1.2->autogluon) (1.2)
Requirement already satisfied: autogluon.multimodal==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon) (1.2)
Requirement already satisfied: autogluon.timeseries==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.timeseries[all]==1.2->autogluon) (1.2)
Requirement already satisfied: numpy<2.1.4,>=1.25.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.26.4)
Requirement already satisfied: scipy<1.16,>=1.5.4 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.13.1)
Requirement already satisfied: scikit-learn<1.5.3,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.5.2)
Requirement already satisfied: networkx<4,>=3.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (3.4.2)
Requirement already satisfied: pandas<2.3.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (2.2.2)
Requirement already satisfied: tqdm<5,>=4.58 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (4.67.1)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (2.32.3)
Requirement already satisfied: matplotlib<3.11,>=3.7.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (3.10.0)
Requirement already satisfied: boto3<2,>=1.18 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.37.1)
Requirement already satisfied: autogluon.common==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.2)
```

Loading the iris dataset and preprocessing

```
import pandas as pd
import autogluon.common as ag
from autogluon.tabular import TabularDataset, TabularPredictor
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

# Load the Iris dataset and construct a DataFrame
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Split the data into train, validation, and test subsets
train_val, test_set = train_test_split(df, test_size=0.1, stratify=df['target'], random_state=42)
train_set, val_set = train_test_split(train_val, test_size=0.2 / 0.9, stratify=train_val['target'], random_state=42)

# Convert DataFrames to Autogluon TabularDataset format
train_data = TabularDataset(train_set)
val_data = TabularDataset(val_set)
test_data = TabularDataset(test_set)
true_labels = test_data['target']
```

Creates a predictor function that fits a tabular predictor using hyperparameter tuning based on the search method provided

```
def build_predictor(search_method: str) -> TabularPredictor:
    """
    Constructs and fits a TabularPredictor using hyperparameter tuning.

    Parameters:
        search_method (str): The search strategy for hyperparameter tuning ('random' or 'bayes').

    Returns:
        A fitted TabularPredictor.
    """
    predictor = TabularPredictor(label='target', problem_type='multiclass', eval_metric='accuracy')
    return predictor.fit(
        train_data=train_data,
        tuning_data=val_data,
        hyperparameters={
            'NN_TORCH': {
                'batch_size': ag.space.Categorical(2, 4),
                'num_epochs': ag.space.Categorical(1, 3, 5),
                'learning_rate': ag.space.Categorical(1e-3, 1e-5),
                'num_layers': 1,
                'hidden_size': 16,
                'activation': 'relu'
            }
        },
        hyperparameter_tune_kwargs={
            'num_trials': 10,
            'scheduler': 'local',
            'searcher': search_method
        },
        num_bag_folds=5,
        num_stack_levels=0,
        use_bag_holdout=True,
        time_limit=100
    )
```

Evaluates the predictor model on the given dataset and shows the metrics and leaderboard stats

```
def evaluate_model(predictor: TabularPredictor, dataset: TabularDataset, description: str):
    """
    Evaluates the predictor on the provided dataset and prints metrics and leaderboard.

    Parameters:
        predictor (TabularPredictor): The model to evaluate.
        dataset (TabularDataset): The dataset for evaluation.
        description (str): A short description to identify the search strategy.
    """
    predictions = predictor.predict(dataset.drop(columns=['target']))
    acc = accuracy_score(true_labels, predictions)
    f1 = f1_score(true_labels, predictions, average='weighted')

    print(description)
    print(f"Test Accuracy: {acc:.4f}")
    print(f"Test F1: {f1:.4f}")

    lb = predictor.leaderboard(dataset)
    print("Leaderboard:")
    print(lb[['model', 'score_test', 'fit_time']])
```

Using the random search strategy

```
# Create and evaluate a predictor using random search
random_pred = build_predictor('random')
evaluate_model(random_pred, test_data, "Random Search:")
```

Outputs

1. System Info

```
===== System Info =====
AutoGluon Version: 1.2
Python Version: 3.11.11
Operating System: Linux
Platform Machine: x86_64
Platform Version: #1 SMP PREEMPT_DYNAMIC Thu Jun 27 21:05:47 UTC 2024
CPU Count: 2
Memory Avail: 10.49 GB / 12.67 GB (82.7%)
Disk Space Avail: 71.16 GB / 107.72 GB (66.1%)
=====
```

```

Autogluon will save models to: /content/AutogluonModels/ag-20250226-183054
Train Data Rows: 104
Train Data Columns: 4
Tuning Data Rows: 15
Tuning Data Columns: 4
Label Column: target
Problem Type: multiclass
Preprocessing data ...
Train Data Class Count: 3
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
Available Memory: 10738.08 MB
Train Data (Original) Memory Usage: 0.00 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the features.
Stage 1 Generators:
  Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
  Fitting FillNaFeatureGenerator...
Stage 3 Generators:
  Fitting IdentityFeatureGenerator...
Stage 4 Generators:
  Fitting DropUniqueFeatureGenerator...
Stage 5 Generators:
  Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
  ('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Types of features in processed data (raw dtype, special dtypes):
  ('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
0.1s = Fit runtime
4 features in original data used to generate 4 features in processed data.
Train Data (Processed) Memory Usage: 0.00 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.11s ...
AutoGluon will gauge predictive performance using evaluation metric: 'accuracy'
To change this, specify the eval_metric parameter of Predictor()
use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).

```

```

User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{'batch_size': Categorical(2, 4), 'num_epochs': Categorical(1, 3, 5), 'learning_rate': Categorical(0.001, 1e-05), 'num_layers': 1, 'hidden_size': 16, 'activation': 'relu'}],
}
Fitting 1 L1 models, fit_strategy="sequential" ...
Hyperparameter tuning model: NeuralNetTorch_BAG_L1 ... Tuning model for up to 89.9s of the 99.89s of remaining time.
+-----+-----+
| Configuration for experiment | NeuralNetTorch_BAG_L1 |
+-----+-----+
| Search algorithm             | BasicVariantGenerator |
| Scheduler                   | FIFOSScheduler        |
| Number of trials             | 10                     |
+-----+-----+

View detailed results here: /content/AutogluonModels/ag-20250226-183054/models/NeuralNetTorch_BAG_L1
Fitted model: NeuralNetTorch_BAG_L1/00804_00000 ...
0.4904 = Validation score (accuracy)
24.18s = Training runtime
0.02s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/00804_00001 ...
0.4135 = Validation score (accuracy)
19.85s = Training runtime
0.04s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/00804_00002 ...
0.6731 = Validation score (accuracy)
21.41s = Training runtime
0.02s = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 99.89s of the 9.36s of remaining time.
Ensemble Weights: {'NeuralNetTorch_BAG_L1/00804_00002': 1.0}
0.6667 = Validation score (accuracy)
0.01s = Training runtime
0.0s = Validation runtime
AutoGluon training complete, total runtime = 90.67s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 780.5 rows/s (15 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/content/AutogluonModels/ag-20250226-183054")
Random Search:
Test Accuracy: 0.6774
Test F1: 0.5484
Leaderboard:

```

	model	score_test	fit_time
0	NeuralNetTorch_BAG_L1/00804_00002	0.677419	21.407012
1	WeightedEnsemble_L2	0.677419	21.412683
2	NeuralNetTorch_BAG_L1/00804_00001	0.451613	19.853127
3	NeuralNetTorch_BAG_L1/00804_00000	0.354839	24.184899

```

# Create and evaluate a predictor using Hyperband Search with Bayesian optimization
bayes_pred = build_predictor('bayes')
evaluate_model(bayes_pred, test_data, "Hyperband Search + Bayes Optimization:")

```

```
Verbosity: 2 (Standard Logging)
AutoGluon Version: 1.2
Python Version: 3.11.11
Operating System: Linux
Platform Machine: amd64
Platform Version: #1 SMP PREEMPT_DYNAMIC Thu Jun 27 21:05:47 UTC 2024
CPU Count: 2
Memory Avail: 16.01 GB / 12.67 GB (79.0%)
Disk Space Avail: 71.16 GB / 107.72 GB (66.1%)

=====
No presets specified! To achieve strong results with AutoGluon, it is recommended to use the available presets. Defaulting to "medium"....
Recommended Presets (for more details refer to https://auto.gluon.ai/tabular/tutorials/tabular/tabular-essentials.html#presets):
presets="experimental" : Best in v1.2: Pre-trained foundation model + parallel fits. The absolute best accuracy without consideration for inference speed. Does not support GPU.
presets="best" : Maximize accuracy. Recommended for most users. Use in competitions and benchmarks.
presets="high" : Strong accuracy with fast inference speed.
presets="good" : Good accuracy with very fast inference speed.
presets="medium" : Fast training time, ideal for initial prototyping.
Warning: Hyperparameter tuning is currently experimental and may cause the process to hang.
Beginning AutoGluon training ... Time limit = 180s
AutoGluon will save models to "/content/AutoGluonModels/ag-20250226_183225"
Train Data Rows: 104
Train Data Columns: 4
Tuning Data Rows: 15
Tuning Data Columns: 4
Label Column: target
Problem Type: multiclass
Preprocessing data ...
Train Data Class Count: 3
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
Available Memory: 10249.98 MB
Train Data (Original) Memory Usage: 0.00 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the features.
Stage 1 Generators:
Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
Fitting FillNAFeatureGenerator...
Stage 3 Generators:
Fitting IdentityFeatureGenerator...
Stage 4 Generators:
Fitting DropNullFeatureGenerator...
Stage 5 Generators:
Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Types of features in processed data (raw dtype, special dtypes):
('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
0.8s = Fit runtime
4 features in original data used to generate 4 features in processed data.
Train Data (Processed) Memory Usage: 0.00 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.06s ...
AutoGluon will guess predictive performance using evaluation metric: 'accuracy'
To change this, specify the eval_metric parameter of Predictor().
use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).
User-specified model hyperparameters to be fit:

use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).
User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{'batch_size': Categorical[2, 4], 'num_epochs': Categorical[1, 3, 5], 'learning_rate': Categorical[0.001, 1e-05], 'num_layers': 1, 'hidden_size': 16, 'activation': 'relu'}],
}
Fitting 11 models, fit_strategy="sequential" ...
Hyperparameter tuning model: NeuralNetTorch_BAG_11 ... Tuning model for up to 89.95s of the 99.94s of remaining time.
+-----+-----+
| Configuration for experiment | NeuralNetTorch_BAG_11 |
+-----+-----+
| Search algorithm | SearchGenerator |
| Scheduler | FIFO Scheduler |
| Number of trials | 10 |
+-----+-----+

View detailed results here: /content/AutoGluonModels/ag-20250226_183225/models/NeuralNetTorch_BAG_11
Fitted model: NeuralNetTorch_BAG_11/849e56c3 ...
0.4904 = Validation score (accuracy)
24.32s = Training runtime
0.02s = Validation runtime
Fitted model: NeuralNetTorch_BAG_11/50862dce ...
0.0192 = Validation score (accuracy)
21.33s = Training runtime
0.02s = Validation runtime
Fitted model: NeuralNetTorch_BAG_11/afff5dab ...
0.625 = Validation score (accuracy)
20.44s = Training runtime
0.03s = Validation runtime
Fitting model: WeightedEnsemble_12 ... Training model for up to 99.94s of the 9.05s of remaining time.
Ensemble weights: {'NeuralNetTorch_BAG_11/afff5dab': 1.0}
0.6 = Validation score (accuracy)
0.0s = Training runtime
0.0s = Validation runtime
AutoGluon training complete, total runtime = 90.98s ... Best model: WeightedEnsemble_12 | Estimated inference throughput: 799.0 rows/s (15 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/content/AutoGluonModels/ag-20250226_183225")

Hyperband Search + Bayes Optimization:
Test Accuracy: 0.5866
Test F1: 0.4920
Leaderboard:
+-----+-----+-----+-----+
| model | score_test | fit_time |
+-----+-----+-----+-----+
0 NeuralNetTorch_BAG_11/afff5dab | 0.58645 | 20.440542 |
1 WeightedEnsemble_12 | 0.58645 | 20.444279 |
2 NeuralNetTorch_BAG_11/849e56c3 | 0.354839 | 24.316839 |
3 NeuralNetTorch_BAG_11/50862dce | 0.032258 | 21.331169 |
```

Some Observations

Impact of Hyperparameters on Model Performance

Epochs: Increasing the number of epochs generally improves model performance because it allows the model more iterations to learn from the data. This trend is visible in the loss-versus-epochs plots, where a higher epoch count corresponds with lower loss values, indicating better learning.

Batch Size: A smaller batch size tends to enhance performance, as it leads to more frequent weight updates during training. In contrast, larger batch sizes may result in higher loss values, suggesting that the model might be less responsive to subtle patterns in the data.

Learning Rate: A higher learning rate can help the model converge faster, often resulting in lower loss values. This relationship is observed across different hyperparameter settings, where increasing the learning rate generally correlates with improved performance.

Manual Tuning vs. Automated Search

Automated Search: Automated hyperparameter tuning is particularly advantageous for large, complex models. It efficiently explores a broad range of parameter combinations, often achieving superior results while reducing the time and effort required compared to manual tuning.

Manual Tuning: Manual tuning can be beneficial when you possess strong domain knowledge and intuition about the problem. It may be a more practical option for smaller models or when computational resources are limited, even though it might not explore the parameter space as thoroughly as automated methods.